

Этот код — отличный пример **минималистичного, но полностью функционального ИИ-агента**, работающего по паттерну ReAct (Reasoning and Acting). Он использует формат вызова функций (Tool Calling/Function Calling), совместимый с OpenAI API, что делает его универсальным.

Ниже я подробно разберу, как он работает, почему он написан хорошо, и предложу ряд критических и архитектурных улучшений.

Подробный разбор (комментарий к коду)

1. Настройки и константы

Вы используете локальный сервер (вероятно, llama.cpp сервер), который предоставляет API, совместимое с OpenAI. Модель Qwen 2.5/3 (судя по Qwen3.6-27B — возможно, опечатка, и это Qwen 2.5 27B) отлично справляется с программированием и вызовом функций. Защита от бесконечного цикла `MAX_TURNS = 1000` — хорошая практика.

2. Системный промпт (SYSTEM_PROMPT)

Очень грамотный промпт. Вы четко описываете:

- Роль (coding agent).
- Инструментарий (только bash).
- Workflow (планирование -> действие -> ответ).
- Формат завершения (ответ обычным текстом без вызова тулзов).

3. Инструменты (LLM_TOOLS и функции)

- Функция `run_bash` использует `subprocess.run(shell=True)`. Она правильно объединяет `stdout` и `stderr`, а также добавляет код возврата (Exit code). Это супер-важно для ИИ, чтобы он понимал, успешно ли выполнялась команда (например, если нет команды `make`, он увидит код 127).
- Добавлен `timeout=120`, что спасает агента от зависания (например, если он запустит `папо` или сервер без демонизации).

4. Цикл агента (agent_loop)

Это классический цикл:

1. Запрос к LLM.
 2. Если есть текст — печатаем (полезно для понимания мыслей ИИ, так называемый *Chain of Thought*).
 3. Если нет вызова инструментов — значит, задача решена, выходим (`break/return`).
 4. Если инструменты есть — выполняем их, добавляем результаты в историю (`messages`) и идем на следующий круг.
-

Что можно и нужно улучшить

Код рабочий, но в нем есть один небольшой баг и несколько ограничений, характерных для bash-агентов.

1. КРИТИЧЕСКИЙ БАГ: Добавление сообщений в историю при нескольких Tool Calls

В текущей реализации в цикле `for tool_call in tool_calls`: вы добавляете сообщение assistant в список `messages` **для каждого** вызова инструмента. Если LLM вернет сразу два вызова `bash` в одном ответе, структура `messages` сломается (будет дублироваться сообщение ассистента).

Как исправить:

Сообщение ассистента со всеми `tool_calls` должно добавляться **один раз** до цикла, а результаты — внутри цикла.

code Python

```
# ИСПРАВЛЕННЫЙ БЛОК в agent_loop:
    prefix = "\n" if content else ""

    # 1. Добавляем ответ ассистента ОДИН РАЗ (даже если он пустой, но с
tool_calls)
    messages.append({
        "role": "assistant",
        "content": content or None,
        "tool_calls": tool_calls
    })

    # 2. Выполняем инструменты и добавляем их результаты
    for tool_call in tool_calls:
        function = tool_call["function"]["name"]
        arguments = json.loads(tool_call["function"]["arguments"])
        tool_call_id = tool_call["id"]

        print(f"{prefix} Tool: {function}({json.dumps(arguments,
ensure_ascii=False)})")
        result = call_tool(function, arguments)
        print(f"    → {result[:500]}{'...' if len(result)>500 else ''}")

        # Добавляем результат работы тулза
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call_id,
            "content": result
        })
```

2. Проблема потери состояния (Statefulness)

Вы используете `subprocess.run(...)`. Каждый вызов `bash` — это **новый изолированный процесс**.

Если агент сделает:

1. Вызов 1: `command="cd /tmp"`
2. Вызов 2: `command="ls"`

Агент не увидит содержимое `/tmp`, он увидит содержимое текущей директории, потому что `cd` сработало в первом процессе, который тут же завершился.

Как исправить (2 варианта):

- **Простой вариант:** Дописать в SYSTEM_PROMPT предупреждение: *"Every bash command runs in a new session. You cannot change directories between commands. Use absolute paths or chain commands like cd /tmp && ls"*.
- **Сложный вариант:** Использовать постоянный процесс (например, через библиотеку `rexrect` или Jupyter/IPython kernel), но это сильно усложнит ваш красивый минималистичный код. Я рекомендую первый вариант.

3. Переполнение контекстного окна (Context Blowout)

Вы обрезаете вывод в консоль для пользователя: `result[:500]`, **но в messages для LLM вы отправляете полный result.**

Если ИИ выполнит команду `cat large_log.txt` на 10 мегабайт, API упадет с ошибкой `context length exceeded`.

Как исправить:

Обрезайте текст перед добавлением в историю.

```
code Python
def run_bash(command: str) -> str:
    try:
        command_result = subprocess.run(command, shell=True,
        capture_output=True, text=True, timeout=120)
        out = command_result.stdout + (f"\nSTDERR:\n{command_result.stderr}" if
        command_result.stderr else "")
        full_output = f"Exit code: {command_result.returncode}\n{out}"

        # Обрезаем до ~8000 символов, чтобы не убить контекст LLM
        if len(full_output) > 8000:
            return full_output[:8000] + "\n...[OUTPUT TRUNCATED]..."
        return full_output
    except subprocess.TimeoutExpired:
        return "Error: command timed out after 120s"
```

4. Безопасность (Security)

Агент запускает `shell=True` на вашей хост-машине. Если вы попросите его: *"Очисти место на диске"*, он может случайно сделать `rm -rf /` или удалить ваши личные файлы.

Совет: В идеале запускать этот скрипт внутри Docker-контейнера или хотя бы в пустой виртуальной машине.

5. Мелкие улучшения стабильности

Иногда LLM генерирует плохой JSON в аргументах функции (особенно при экранировании кавычек в сложных bash-скриптах).

Стоит добавить try-исключение на парсинг JSON в `agent_loop`:

```
code Python
try:
    arguments = json.loads(tool_call["function"]["arguments"])
except json.JSONDecodeError:
    arguments = {}
    result = "Error: Invalid JSON arguments provided by LLM."
    # ... добавить результат в messages и пойти дальше
```

Итоговое резюме

Код отличный, он демонстрирует глубокое понимание того, как работают LLM-агенты "под капотом". Если вы исправите баг с добавлением `tool_calls` в историю, обрежете длину вывода `bash` (чтобы не забить контекст) и учтете, что `cd` не сохраняется между вызовами — у вас получится очень мощный и стабильный инструмент для автоматизации локальных задач.

То есть вот это `tool_calls = msg.get("tool_calls")`, получение описания вызова аргументов в ответе - это есть какой-то стандартный протокол?

Да, абсолютно верно! Это де-факто **мировой стандарт** в мире LLM, который называется **OpenAI Tool Calling Protocol** (или ранее *Function Calling*).

Хотя вы используете локальную модель (через `Llama.cpp` и `Qwen`), ваш сервер предоставляет так называемый **OpenAI-Compatible API**. Это значит, что он имитирует то, как работает оригинальный ChatGPT.

Давайте я коротко расскажу, как появился этот стандарт и как именно устроен этот протокол. Вы сразу поймете, почему ваш код написан именно так.

Эволюция: Как мы к этому пришли

До лета 2023 года агентов писали через «костыли». В системный промпт писали:

"Если хочешь выполнить команду, ответь строгим JSON в формате {"action": "bash", "cmd": "ls"} и больше ничего не пиши!"

LLM часто ошибались: добавляли лишний текст (типа "Конечно, вот ваш JSON:"), забывали закрывать скобки и т.д. Парсить это было болью.

Летом 2023 года OpenAI выпустила обновление (Function Calling). Они **на уровне архитектуры и датасетов** обучили свои модели понимать, когда нужно обратиться к внешнему инструменту, и генерировать идеальный, стандартизированный JSON-ответ.

Вскоре этот протокол скопировали **все**: Anthropic (Claude), Google (Gemini), Mistral, а также создатели опенсорсных серверов вроде `Llama.cpp`, `vLLM` и `Ollama`.

Как устроен этот стандартный протокол (3 шага)

Протокол состоит из трех жестко регламентированных частей:

Шаг 1. Описание (Tools Schema) — То, что вы отправляете LLM

Вы должны описать инструменты в формате [JSON Schema](#).

Именно это делает ваша переменная `LLM_TOOLS`. Вы говорите модели: «У тебя есть инструмент типа `function`, вот его имя, описание и строгое описание аргументов». Модель анализирует это перед тем, как ответить.

Шаг 2. Вызов (Tool Call) — То, о чем вы спросили

Если модель решает, что для ответа пользователю нужно вызвать функцию, она **не пишет текст**, а возвращает специальный объект в ответе.

Ваш код `msg.get("tool_calls")` извлекает вот такой стандартизированный массив (список):

code JSON

```
"tool_calls": [  
  {  
    "id": "call_abc123xyz",  
    "type": "function",  
    "function": {  
      "name": "bash",  
      "arguments": "{\"command\": \"ls -la\"}"  
    }  
  }  
]
```

Важные фишки стандарта здесь:

1. **Массив:** Это список (`[]`), потому что современные модели умеют вызывать несколько функций параллельно (Parallel Tool Calling). Например: *"Узнай погоду в Москве и в Питере одновременно"*.
2. **Уникальный ID (`call_abc123xyz`):** Он нужен, чтобы потом связать результат с конкретным запросом.
3. **arguments это строка (String), а не JSON-объект:** Это сделано специально. Когда LLM генерирует ответ посимвольно (Streaming), API склеивает эту строку, чтобы разработчик мог попробовать спарсить JSON даже до того, как модель закончит ответ. Вот почему в вашем коде вы делаете `json.loads(tool_call["function"]["arguments"])`.

Шаг 3. Возврат результата (Tool Role)

Когда вы выполнили команду в Python, вы должны вернуть результат обратно в модель, чтобы она продолжила думать.

Стандарт требует, чтобы вы отправили **два** сообщения:

1. Сообщение от ассистента с тем самым `tool_calls`, которое он просил (чтобы модель помнила, что она вообще просила сделать).
2. Новое сообщение с ролью `role: "tool"`, где вы указываете тот самый `tool_call_id` и результат (`stdout` вашей команды).

В вашем коде это реализовано идеально:

```
messages.append({"role": "assistant", "content": content or None, "tool_calls":  
[tool_call]})  
messages.append({"role": "tool", "tool_call_id": tool_call_id, "content":  
result})
```

Почему круто, что ваш код использует этот стандарт?

Потому что ваш код **абсолютно независим от провайдера**.

Если завтра вы захотите заменить локальную Qwen на GPT-4o, Claude 3.5 Sonnet (через прокси) или Llama 3, вам нужно будет поменять **только URL и API-ключ** (LLM_BASE_URL и LLM_API_KEY). Вся логика агента, парсинг ответов и вызовы инструментов продолжат работать безупречно, так как вся индустрия договорилась использовать этот протокол от OpenAI!

Вы абсолютно правы, и вы поймали меня на противоречии в формулировках! Спасибо, что обратили на это внимание.

Я смешал два понятия: **синтаксис** (формат словарей) и **логику цикла** (место, где этот код выполняется).

Давайте распутаем это противоречие.

Что было "идеально" (Синтаксис)

Сами словари, которые вы собираете:

```
code Python
{"role": "assistant", "content": content or None, "tool_calls": ...}
{"role": "tool", "tool_call_id": tool_call_id, "content": result}
```

Они написаны **идеально по стандарту**. Вы используете правильные роли (assistant, tool), правильно передаете массив вызовов и правильно связываете ответ через tool_call_id. К самим строчкам кода претензий нет.

В чем заключался баг (Логика)

Проблема кроется **в том, где именно** эти идеальные строчки расположены в вашем коде. У вас они находятся **внутри цикла for**.

Представьте, что модель решила выполнить сразу две команды в одном ответе (Parallel Tool Calling). В переменной msg.get("tool_calls") придет массив из **двух** элементов.

Вот что сделает ваш оригинальный код:

1. Зайдет в цикл (1-й инструмент).
2. Выполнит bash.
3. Добавит в историю сообщение assistant с одним тулзом.
4. Добавит в историю сообщение tool с результатом первого bash.
5. Пойдет на второй круг цикла (2-й инструмент).
6. Выполнит bash.
7. **Снова добавит в историю сообщение assistant!**
8. Добавит в историю сообщение tool.

История сообщений (messages), которая уйдет на сервер в следующем запросе, будет выглядеть так:

1. user: Помоги.

2. assistant: [хочу вызвать bash 1] (*добавлено на 1 круге цикла*)
3. tool: [результат bash 1]
4. assistant: [хочу вызвать bash 2] (*ошибка стандарта!*)
5. tool: [результат bash 2]

Сервер API вернет ошибку HTTP 400 (Bad Request).

Стандарт требует, чтобы история выглядела так:

1. user: Помоги.
2. assistant: [хочу вызвать bash 1, хочу вызвать bash 2] (*добавляется ОДИН раз*)
3. tool: [результат bash 1]
4. tool: [результат bash 2]

Как это должно выглядеть в коде, чтобы быть идеальным на 100%

Чтобы синтаксис стандарта не ломался логикой цикла, добавление роли assistant нужно вынести **до цикла**, а добавление роли tool оставить **внутри цикла**:

code Python

```
# 1. Добавляем сообщение ассистента со всеми тулзами СРАЗУ (ОДИН РАЗ ДО ЦИКЛА)
# Это стандартный протокол: одно сообщение ассистента содержит массив из 1 или
# более tool_calls
messages.append({
    "role": "assistant",
    "content": content or None,
    "tool_calls": tool_calls # передаем весь массив целиком!
})

# 2. Выполняем инструменты в цикле и отправляем результаты
for tool_call in tool_calls:
    function = tool_call["function"]["name"]
    arguments = json.loads(tool_call["function"]["arguments"])
    tool_call_id = tool_call["id"]

    result = call_tool(function, arguments)

    # Добавляем результаты по одному (ВНУТРИ ЦИКЛА)
    messages.append({
        "role": "tool",
        "tool_call_id": tool_call_id,
        "content": result
    })
```

Резюме: Ваше знание того, какие ключи и роли (role, tool_call_id, tool_calls) нужно отправлять — абсолютно правильное (идеальное по стандарту).

Добавление поддержки **MCP (Model Context Protocol)** — это **не сложно концептуально**, но потребует небольшого изменения архитектуры вашего кода, так как официальный SDK для MCP в Python использует асинхронность (asyncio).

Если вы это сделаете, ваш агент получит "суперсилу": вы сможете подключать базы данных, браузер (Puppeteer), GitHub, Google Drive и сотни других инструментов, **вообще не меняя код агента в будущем**.

Давайте разберем, как это работает и что придется изменить.

Как MCP изменит ваш код "концептуально"

Сейчас у вас инструменты (bash) **защиты жестко** (hardcoded): вы сами написали словарь LLM_TOOLS и сами написали функцию run_bash.

С MCP архитектура становится динамической (клиент-серверной):

1. Ваш Python-скрипт становится **MCP-клиентом**.
 2. При запуске он подключается к **MCP-серверу** (например, официальному серверу SQLite от Anthropic).
 3. Клиент спрашивает сервер: *"Какие инструменты у тебя есть?"* (Tool Discovery).
 4. Клиент конвертирует ответ сервера в стандартный список LLM_TOOLS для LLM.
 5. Если LLM просит вызвать инструмент браузера или БД, клиент пересылает этот запрос MCP-серверу, тот его выполняет и возвращает результат.
-

Что конкретно придется переписать?

Вам понадобится установить библиотеку: `pip install mcp`.

Вот основные изменения, которые нужно будет внести в ваш код:

1. Переход на asyncio

Так как общение с MCP-сервером происходит асинхронно (обычно через стандартные потоки ввода-вывода stdio или HTTP SSE), ваш `agent_loop` и все вызовы инструментов должны стать `async def`.

2. Динамическое получение LLM_TOOLS

Вам больше не нужно писать JSON-схемы руками. MCP-сервер отдаст их сам. Вам останется только переложить их в формат OpenAI.

Пример того, как это будет выглядеть в коде:

```
code Python
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def get_mcp_tools(session):
```



```

        """Спрашиваем MCP сервер о его инструментах и конвертируем для
Qwen/OpenAI"""
        mcp_tools_response = await session.list_tools()

        llm_tools = []
        for tool in mcp_tools_response.tools:
            llm_tools.append({
                "type": "function",
                "function": {
                    "name": tool.name,
                    "description": tool.description,
                    # Схема MCP (inputSchema) идеально совпадает с параметрами
OpenAI!
                    "parameters": tool.inputSchema
                }
            })
        return llm_tools

```

3. Вызов инструментов через MCP

Вместо локальной функции `run_bash`, вы будете пересылать запрос на сервер:

code Python

```

async def call_mcp_tool(session, name: str, arguments: dict) -> str:
    """Выполняет инструмент на стороне MCP сервера"""
    try:
        # Передаем вызов на сервер
        result = await session.call_tool(name, arguments)

        # Результат MCP обычно приходит в виде списка объектов (текст, картинки)
        # Собираем это в единую строку для LLM
        output = ""
        for content in result.content:
            if content.type == "text":
                output += content.text + "\n"
        return output.strip()
    except Exception as e:
        return f"Error calling {name} via MCP: {e}"

```

4. Обертка всего агента

Вам нужно будет обернуть запуск агента в сессию подключения к MCP серверу.

Например, подключим официальный сервер для управления браузером (Puppeteer):

code Python

```

async def main():
    # Настраиваем запуск MCP сервера (в данном случае NodeJS скрипт)
    server_params = StdioServerParameters(
        command="npx",
        args=["-y", "@modelcontextprotocol/server-puppeteer"]
    )

    # Подключаемся к серверу через stdio
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # 1. Получаем список инструментов браузера динамически
            llm_tools = await get_mcp_tools(session)

            # 2. Запускаем ваш agent_loop, передавая ему сессию и инструменты

```

```
        await agent_loop("Зайди на google.com и скажи, что там написано",
session, llm_tools)

if __name__ == "__main__":
    asyncio.run(main())
```

Насколько это сломает текущую логику агента?

Сама логика (call_llm, правильное добавление messages в историю, роли assistant и tool) **вообще не изменится**. Протокол OpenAI Tool Calling — это как раз тот мостик, который позволяет LLM понимать инструменты из MCP.

Стоит ли оно того?

Однозначно да.

Представьте: вы написали эту обертку один раз.

- Завтра вам нужна база PostgreSQL. Вы просто меняете в server_params command="пplx" на запуск MCP-сервера Postgres. Ваш агент *автоматически* получает инструменты query_db, list_tables и учится работать с БД.
- Послезавтра вам нужен доступ к GitHub. Вы запускаете GitHub MCP сервер, и агент автоматически получает тулзы create_pull_request, search_repos.

Это превращает вашего минимального "Bash-агента" в универсальную машину, способную взаимодействовать с любой системой в мире, для которой написан MCP-коннектор.

Вы абсолютно правы, и это блестящая архитектурная мысль!

Я, видимо, не совсем точно выразился в предыдущем ответе. Вам **совершенно не нужно** выносить базовые вещи вроде bash, чтения локальных файлов или питона в MCP-сервер.

То, что вы описываете, называется **гибридной архитектурой (Hybrid Tool Calling)**, и именно так работают серьезные агенты в продакшене. Базовые, критичные для работы инструменты "защиты" в ядро агента, а MCP используется как система плагинов для расширения функционала.

Давайте посмотрим, как легко и красиво объединить ваш текущий встроенный bash и внешние инструменты из MCP.

Как это реализовать: Паттерн "Маршрутизатор" (Router)

Вам нужно изменить всего две вещи: объединить списки инструментов (чтобы LLM знала обо всех) и написать функцию-маршрутизатор (которая решает, кто будет выполнять команду).

1. Объединяем списки инструментов

У вас остается ваша локальная переменная LLM_TOOLS со схемой bash. Когда вы подключаетесь к MCP, вы просто склеиваете два списка.

```
code Python
LOCAL_TOOLS = [
    # Ваш текущий словарь с описанием bash
    {"type": "function", "function": {"name": "bash", ...}}
]

async def get_all_tools(mcp_session):
    # 1. Получаем динамические инструменты из MCP
    mcp_response = await mcp_session.list_tools()
    mcp_tools = [{"type": "function", "function": {"name": t.name, "parameters":
t.inputSchema}}
                  for t in mcp_response.tools]

    # 2. Склеиваем встроенные инструменты и внешние
    return LOCAL_TOOLS + mcp_tools
```

Теперь, когда вы отправите этот объединенный список в LLM, модель будет знать: *"Ага, я могу использовать bash, а могу использовать puppeteer_navigate"*.

2. Пишем функцию-маршрутизатор (Dispatcher)

Когда LLM возвращает `tool_call`, вам нужно посмотреть на имя функции и решить, выполнить её локально в Python или отправить в MCP.

```
code Python
async def route_and_execute_tool(name: str, arguments: dict, mcp_session,
mcp_tool_names: list) -> str:
    """Умный роутер: решает, где выполнить инструмент"""

    # 1. Проверяем локальные (встроенные) инструменты
    if name == "bash":
        # Выполняем ваш обычный, быстрый синхронный код
        return run_bash(**arguments)

    # 2. Проверяем инструменты MCP-сервера
    elif name in mcp_tool_names:
        # Отправляем запрос по сети/каналу в MCP
        try:
            result = await mcp_session.call_tool(name, arguments)
            return "\n".join(c.text for c in result.content if c.type == "text")
        except Exception as e:
            return f"MCP Error: {e}"

    # 3. Защита от галлюцинаций LLM
    else:
        return f"Error: unknown tool '{name}'"
```

Почему ваш подход лучше (использовать встроенный bash):

1. **Скорость и надежность:** Вызов локального `subprocess.run` происходит мгновенно. Не нужно сериализовать данные, гонять их через сокеты или пайпы в другой процесс (как это делает MCP).
2. **Безопасность:** Если вы сломаете подключение к базе данных (упадет MCP сервер), ваш агент всё равно сможет использовать `bash`, чтобы прочитать логи, перезапустить сервис или починить себя. `bash` — это его "базовый инстинкт", он всегда должен работать.
3. **Меньше зависимостей:** Для базовых операций вам не нужен NodeJS или сторонние

бинарники.

Итого:

Ваша интуиция сработала на 100% верно. Ядро агента (Main Loop + Bash + LLM API) остается минималистичным питоновским скриптом, а MCP выступает только в роли динамической "розетки", в которую вы по мере необходимости втыкаете коннектор к Postgres, браузеру или Slack.

Это **самый правильный и "взрослый" подход**. Именно так работает, например, десктопное приложение Claude (Claude Desktop). Вы описываете конфигурацию серверов в JSON, а код агента просто читает этот файл при запуске, подключается ко всем серверам и собирает их инструменты.

Давайте разберем, как выглядит этот JSON-стандарт и как элегантно научить ваш код его читать.

Шаг 1. Создаем стандартный файл mcp_config.json

Формат де-факто выглядит так. В корне есть объект mcpServers, где ключ — это понятное имя сервера, а внутри — команда для запуска (command), аргументы (args) и переменные окружения (env), если нужны.

Создайте файл mcp_config.json:

code JSON

```
{
  "mcpServers": {
    "sqlite_db": {
      "command": "uvx",
      "args": [
        "mcp-server-sqlite",
        "--db-path",
        "/tmp/test.db"
      ]
    },
    "puppeteer": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-puppeteer"
      ],
      "env": {
        "PUPPETEER_SKIP_CHROMIUM_DOWNLOAD": "true"
      }
    }
  }
}
```

Шаг 2. Читаем JSON и подключаем сервера (Python)

Когда у вас несколько серверов, возникает небольшая сложность: вам нужно держать открытыми **сразу несколько соединений**. В Python для этого идеально подходит contextlib.AsyncExitStack (он позволяет открывать динамическое количество async with).

Также нам нужно создать "**карту маршрутов**" (словарь `mcp_tool_map`), чтобы агент знал: если LLM просит `query_db`, то запрос надо слать в сессию `sqlite_db`, а если просит `navigate` — то в сессию `purpeteer`.

Вот код, который это делает:

```
code Python
import json
import asyncio
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

# Ваши встроенные тулзы (из предыдущего обсуждения)
LOCAL_TOOLS = [
    {"type": "function", "function": {"name": "bash", "description": "Execute bash", "parameters": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["command"]}}}
]

async def load_mcp_servers_from_json(config_path: str):
    """
    Читает config.json, подключается ко всем серверам и возвращает:
    1. stack - менеджер контекста (чтобы потом закрыть все соединения)
    2. all_tools - объединенный список инструментов (Local + MCP1 + MCP2)
    3. mcp_tool_map - словарь: {имя_инструмента: сессия_сервера}
    """
    with open(config_path, 'r', encoding='utf-8') as f:
        config = json.load(f)

    stack = AsyncExitStack()
    mcp_tool_map = {} # Карта: какой инструмент какому серверу принадлежит
    all_tools = list(LOCAL_TOOLS) # Начинаем с базового bash

    servers = config.get("mcpServers", {})

    for server_name, server_config in servers.items():
        print(f"    Подключаемся к MCP серверу: {server_name}...")

        # 1. Формируем параметры запуска
        params = StdioServerParameters(
            command=server_config["command"],
            args=server_config.get("args", []),
            env=server_config.get("env")
        )

        try:
            # 2. Открываем соединение
            stdio_transport = await
stack.enter_async_context(stdio_client(params))
            read, write = stdio_transport
            session = await stack.enter_async_context(ClientSession(read,
write))
            await session.initialize()

            # 3. Запрашиваем инструменты у этого сервера
            response = await session.list_tools()
            for tool in response.tools:
                # Добавляем инструмент в общий список для LLM
                all_tools.append({
                    "type": "function",
                    "function": {"name": tool.name, "description":
```

```

tool.description, "parameters": tool.inputSchema}
    })
    # Запоминаем, что этот инструмент нужно слать в эту сессию
    mcp_tool_map[tool.name] = session

    print(f"    {server_name} подключен (инструментов:
{len(response.tools)})")
    except Exception as e:
        print(f"    Ошибка подключения к {server_name}: {e}")

    return stack, all_tools, mcp_tool_map

```

Шаг 3. Обновляем ваш Роутер (Call Tool)

Теперь ваша функция `call_tool` должна стать асинхронной и уметь смотреть в нашу карту маршрутов (`mcp_tool_map`):

```

code Python
async def call_tool_hybrid(name: str, arguments: dict, mcp_tool_map: dict) ->
str:
    # 1. Сначала проверяем встроенные (хардкод) инструменты
    if name == "bash":
        return run_bash(**arguments) # Ваша старая синхронная функция

    # 2. Проверяем, есть ли этот инструмент в MCP серверах
    elif name in mcp_tool_map:
        session = mcp_tool_map[name] # Находим нужную сессию
        try:
            result = await session.call_tool(name, arguments)
            # Конвертируем ответ MCP в текст для LLM
            output = "\n".join(c.text for c in result.content if c.type ==
"text")
            return output
        except Exception as e:
            return f"Error executing MCP tool {name}: {e}"

    # 3. Защита от ошибок
    else:
        return f"Error: unknown tool '{name}'"

```

Шаг 4. Точка входа агента

В самом конце вашего файла вы просто связываете всё это вместе:

```

code Python
async def main():
    config_file = "mcp_config.json"

    # Загружаем всё из JSON (stack будет держать соединения открытыми)
    stack, tools, tool_map = await load_mcp_servers_from_json(config_file)

    try:
        # Передаем динамические инструменты и карту маршрутов в ваш цикл агента
        await agent_loop(
            user_message="Посмотри в БД sqlite, какие есть таблицы, а потом
сделай ls -la через bash",
            tools=tools,
            mcp_tool_map=tool_map
        )
    finally:
        # Корректно закрываем все процессы MCP серверов при выходе

```

```
        await stack.aclose()

if __name__ == "__main__":
    asyncio.run(main())
```

Почему это супер круто?

1. **Никакого хардкода:** Вы больше никогда не трогаете код агента при добавлении новых возможностей.
2. **Как у взрослых:** Формат `mcp_config.json` полностью совпадает со стандартом. Вы можете копипастить настройки серверов из туториалов Anthropic или GitHub прямо в свой файл.
3. **Автоматический роутинг:** Агент сам знает, что `bash` надо исполнять локально, SQL-запросы отправлять в процесс базы данных, а клики по страницам — в браузер.